Google Cloud | Gemini Enterprise

Hackathon Developer Guide: Building the FINTRAC AI Anomaly Detector

Target Platform: Gemini Enterprise, Google Cloud, BigQuery **Focus:** High-code Agent Development, ReAct Architecture, Model Context Protocol (MCP)

1. Hackathon Objective

Welcome to the FINTRAC Agent Hackathon. Your mission is to build an enterprise-grade AI agent capable of detecting financial crime. Traditional Anti-Money Laundering (AML) systems rely on rigid, rule-based SQL logic that generates massive false positives and misses sophisticated, evolving evasion tactics.

In this hackathon, you will leverage the Agent Development Kit (ADK) and Gemini Enterprise to build an intelligent agent that parses raw financial transactions, reasons over patterns using an LLM, autonomously queries a BigQuery database using the Model Context Protocol (MCP), and outputs highly contextual alerts for human compliance analysts.

Key Deliverable: A fully functioning, containerized AI agent that accurately flags "Structuring" (smurfing) and "Velocity" (rapid cross-border multi-hops) anomalies from a synthetic daily batch dataset.

2. Architecture Overview

You are building a secure, serverless pipeline on Google Cloud. This architecture provides the strict audit trail required for federal financial workloads.

Note on Data Residency: While production Canadian federal workloads should target the `northamerica-northeast1` (Montreal) region for sovereign data compliance, hackathon project environments may have quota restrictions. This guide defaults to `us-central1` to ensure availability of the Gemini 2.5 Flash model during the event.

[Synthetic Daily Batch Upload] → [BigQuery Transactions Table] ↓ [Cloud Run: Gemini High-Code Agent] ↓ (MCP Tool Execution) [BigQuery Anomalies Table] → [Looker Studio Analyst Dashboard]

- **BigQuery:** Serves as both the source of truth for raw transaction logs and the destination for flagged anomalies.
- **Gemini / Vertex AI:** Provides the ReAct reasoning engine.

- **Cloud Run:** Hosts the containerized high-code agent and MCP server.
- **Looker Studio:** The zero-code frontend where analysts review the agent's findings on an interactive geographical and flow map.

3. Prerequisites & Environment Setup

Before coding, ensure your team has access to a Google Cloud Project with the Vertex AI and BigQuery APIs enabled.

Step 3.1: Scaffold the Project

Initialize your workspace using the Agents CLI and install the required BigQuery dependencies.

Bash

```
None
# Install the Agents CLI
uvx google-agents-cli setup

# Scaffold a new high-code agent
agents-cli create fintrac-detector -y --deployment-target
agent_runtime
cd fintrac-detector

# Add BigQuery to the project dependencies (Critical for Cloud Run
deployment)
uv add google-cloud-bigquery
```

4. Data Engineering: Ingesting the Batch

You cannot test an AML agent without realistic data. Use this Python script to generate a daily batch of synthetic transactions.

Step 4.1: Generate Synthetic Data

Create `generate_batch.py` in your root folder:

Python

```
None
import csv
import uuid
from datetime import datetime, timedelta
import random

FILENAME = 'fintrac_batch.csv'
BASE_TIME = datetime.now()
LOCAL = ["K1A 0G9", "K2P 2N2", "K1S 5B6"]
OFFSHORE = ["KY1-1104", "BM HM 11", "VG1110"]

transactions = []

# Generate Normal Noise
for i in range(50):
    transactions.append({
        "tx_id": f"TXN-{uuid.uuid4().hex[:8].upper()}",
        "timestamp": (BASE_TIME -
timedelta(minutes=random.randint(1, 1400))).isoformat(),
        "sender": f"ACCT-{random.randint(10000, 99999)}",
        "receiver": f"ACCT-{random.randint(10000, 99999)}",
        "amount": round(random.uniform(50.0, 4500.0), 2),
        "sender_postal": random.choice(LOCAL),
        "receiver_postal": random.choice(LOCAL)
    })

# Inject Structuring Anomaly
```

```
smurf_time = BASE_TIME - timedelta(hours=2)
for i in range(3):
    transactions.append({
        "tx_id": f"TXN-{uuid.uuid4().hex[:8].upper()}",
        "timestamp": (smurf_time +
timedelta(minutes=i*45)).isoformat(),
        "sender": "ACCT-SMURF01",
        "receiver": "ACCT-TARGET99",
        "amount": round(random.uniform(9500.0, 9950.0), 2),
        "sender_postal": "K1S 5B6",
        "receiver_postal": "K2P 2N2"
    })

transactions.sort(key=lambda x: x["timestamp"])

with open(FILENAME, mode='w', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=transactions[0].keys())
    writer.writeheader()
    writer.writerows(transactions)
```

Step 4.2: Load to BigQuery

Bash

```
None
python generate_batch.py
bq mk fintrac_prod
bq load --source_format=CSV --autodetect fintrac_prod.transactions
./fintrac_batch.csv
```

5. Core Agent Development

Important: To prevent `ModuleNotFoundError` during container build, ensure your tool files and instructions are saved directly inside the `app/` folder.

Step 5.1: Build the BigQuery MCP Server (`app/mcp_bigquery.py`)

Python

```
None
from google.cloud import bigquery

client = bigquery.Client()

def query_transactions(sql_query: str):
    """Executes a read-only SQL query against the BigQuery
transactions table."""
    if any(keyword in sql_query.upper() for keyword in ["DROP",
"DELETE", "UPDATE", "INSERT"]):
        return "Error: Only SELECT queries are allowed via this
tool."

    results = client.query(sql_query).result()
    return [dict(row) for row in results]

def log_anomaly(tx_id: str, timestamp: str, source: str, target:
str, amount: float, anomaly_type: str, score: float):
    """Logs a detected suspicious transaction into the
flagged_anomalies table."""
    query = f"""
        INSERT INTO `fintrac_prod.flagged_anomalies`
        (tx_id, timestamp, source, target, amount, anomaly_type,
score)
        VALUES ('{tx_id}', '{timestamp}', '{source}', '{target}',
{amount}, '{anomaly_type}', {score})
    """
    client.query(query).result()
    return f"Successfully logged anomaly for transaction {tx_id}"
```

Step 5.2: Define the Agent Logic (`app/agent.py`)

Python

```
None
import os
import google.auth
from google.adk.agents import Agent
from google.adk.apps import App
from google.adk.models import Gemini
from google.genai import types

from .mcp_bigquery import query_transactions, log_anomaly

_, project_id = google.auth.default()
os.environ["GOOGLE_CLOUD_PROJECT"] = project_id
os.environ["GOOGLE_CLOUD_LOCATION"] = "us-central1"
os.environ["GOOGLE_GENAI_USE_VERTEXAI"] = "True"

current_dir = os.path.dirname(__file__)
instructions_path = os.path.join(current_dir, "GEMINI.md")
with open(instructions_path, "r") as f:
    system_instructions = f.read()

root_agent = Agent(
    name="fintrac_agent",
    model=Gemini(
        model="gemini-2.5-flash",
        retry_options=types.HttpRetryOptions(attempts=3),
    ),
    instruction=system_instructions,
    tools=[query_transactions, log_anomaly],
)

app = App(root_agent=root_agent, name="app")
```

Step 5.3: Define System Instructions (`app/GEMINI.md`)

Plaintext

```
None
You are the FINTRAC Compliance AI Agent. Analyze daily batches and
flag AML patterns.

Rules:
1. Schema Discovery: Always run `SELECT column_name FROM
fintrac_prod.INFORMATION_SCHEMA.COLUMNS WHERE table_name =
'transactions'` before your audit.
2. Structuring: Look for deposits just below $10,000.
3. Velocity: Flag rapid hops to offshore postal codes (e.g., KY,
BM, VG).
4. Use `query_transactions` to audit and `log_anomaly` to report
findings.
```

6. Local Testing & Production Deployment

Test your agent locally using the playground. If you encounter an "address already in use" error, clear the port first.

Bash

```
None
# Run the local testing server
agents-cli playground

# Use this command to clear frozen server ports
fuser -k 8080/tcp
```

Once the agent successfully logs anomalies to BigQuery, deploy it to Cloud Run:

Bash

```
None
agents-cli deploy --project hso-hackathon
```

7. Looker Studio Visualization (The Analyst Dashboard)

To build a compelling presentation, visualize the outputs using Looker Studio's BigQuery connector. Use this exact Custom Query to blend your raw data with the agent's findings:

SQL

```
None
SELECT
  a.timestamp,
  a.tx_id,
  a.source,
  a.target,
  a.amount,
  a.anomaly_type,
  a.score,
  t.sender_postal,
  t.receiver_postal,
  CASE
    WHEN t.receiver_postal LIKE 'KY%' THEN 'Cayman Islands'
    WHEN t.receiver_postal LIKE 'BM%' THEN 'Bermuda'
    WHEN t.receiver_postal LIKE 'VG%' THEN 'British Virgin Islands'
    ELSE 'Canada'
  END as destination_country
FROM `fintrac_prod.flagged_anomalies` a
LEFT JOIN `fintrac_prod.transactions` t ON a.tx_id = t.tx_id
```

Dashboard Components to Build:

1. **The Flow Map (Community Sankey Diagram):** Set Dimension 1 to `source`, Dimension 2 to `target`, weighted by `amount`. Thick lines indicate the volume of CAD moving through mule accounts.
2. **Geographic Risk Map (Google Maps Bubble):** Plot `destination_country` and color-code the pins by the AI's confidence `score`. Highlights rapid movement from Canada to offshore jurisdictions.
3. **Analyst Audit Table:** A flat table containing the exact timestamps, transaction IDs, and AI Confidence Scores (e.g., 0.98) for fast exporting into a Suspicious Transaction Report (STR). Ensure cross-filtering is enabled across all charts.